Lab 7: Fuzzing for Input Validation Bugs

Name: **Balla Santhosh** SRN: **PES1UG24AM063** Section: B

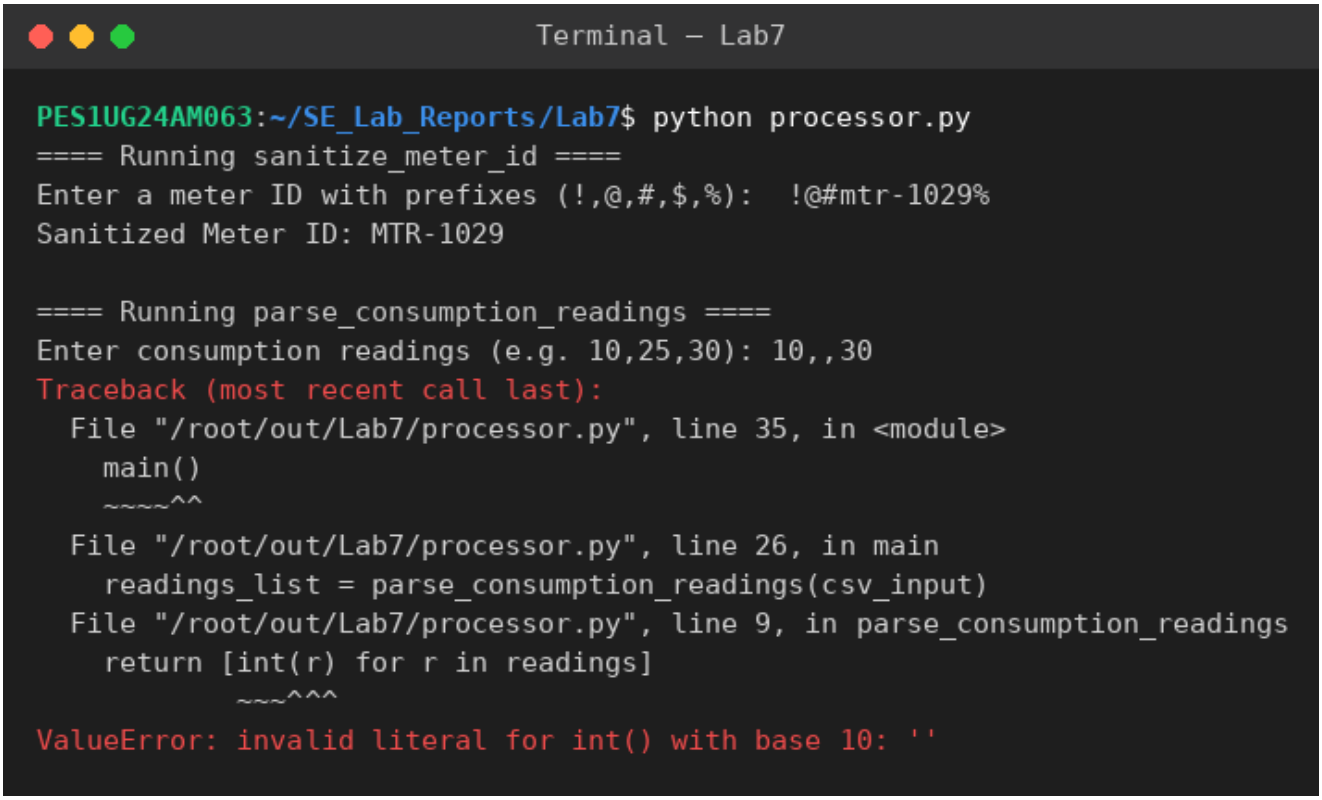
Assigned repo: L6_L7_01_water_utility_billing Tool: Hypothesis (property-based fuzzing) + pytest

Step 1: Dependencies

`pip install hypothesis` and `pip install pytest`

Step 2: Running the buggy processor.py

Command: `python processor.py`. The meter ID is sanitised correctly, but entering a CSV with an empty element (`10,,30`) crashes the program with a ValueError before the third function is reached.

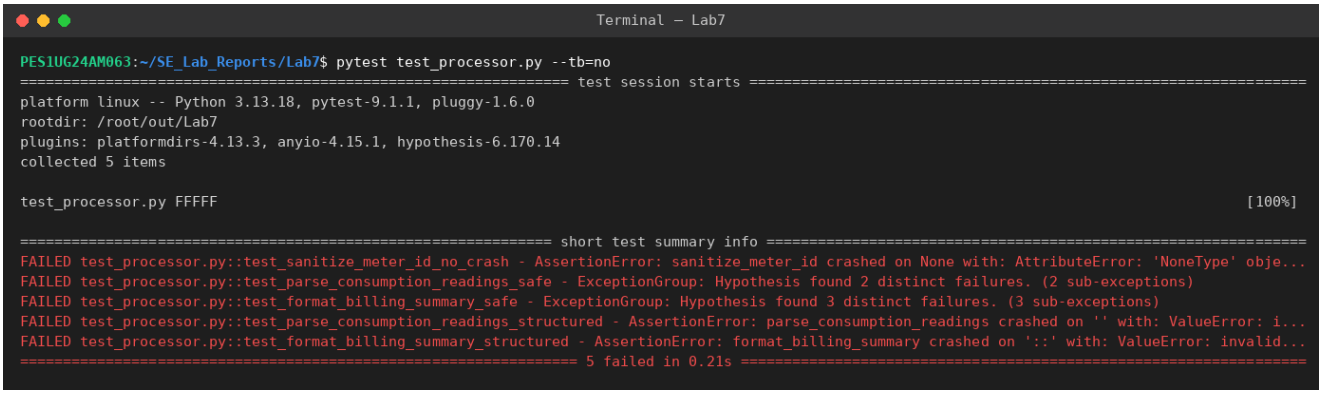


Step 3: Completing test_processor.py with Hypothesis

Each of the three starter tests feeds `st.text() | st.none()` to the function inside a try/except and fails with the offending input if any exception escapes. On top of 'no crash', each test checks a property of the result (sanitised IDs contain no !@#$% and no surrounding whitespace; readings are a list of ints; summaries are strings). Two extra structured fuzz tests build realistic inputs (comma-separated integers mixed with junk tokens, and Name:Usage:Zone records with bad usage fields) to check that valid data is still parsed correctly.

Step 4: Fuzz tests failing on the buggy code

Command: `pytest test_processor.py` (shown with `--tb=no` so the summary fits on one screen). All 5 tests fail.



Bugs discovered by Hypothesis (shrunk to the smallest failing case):

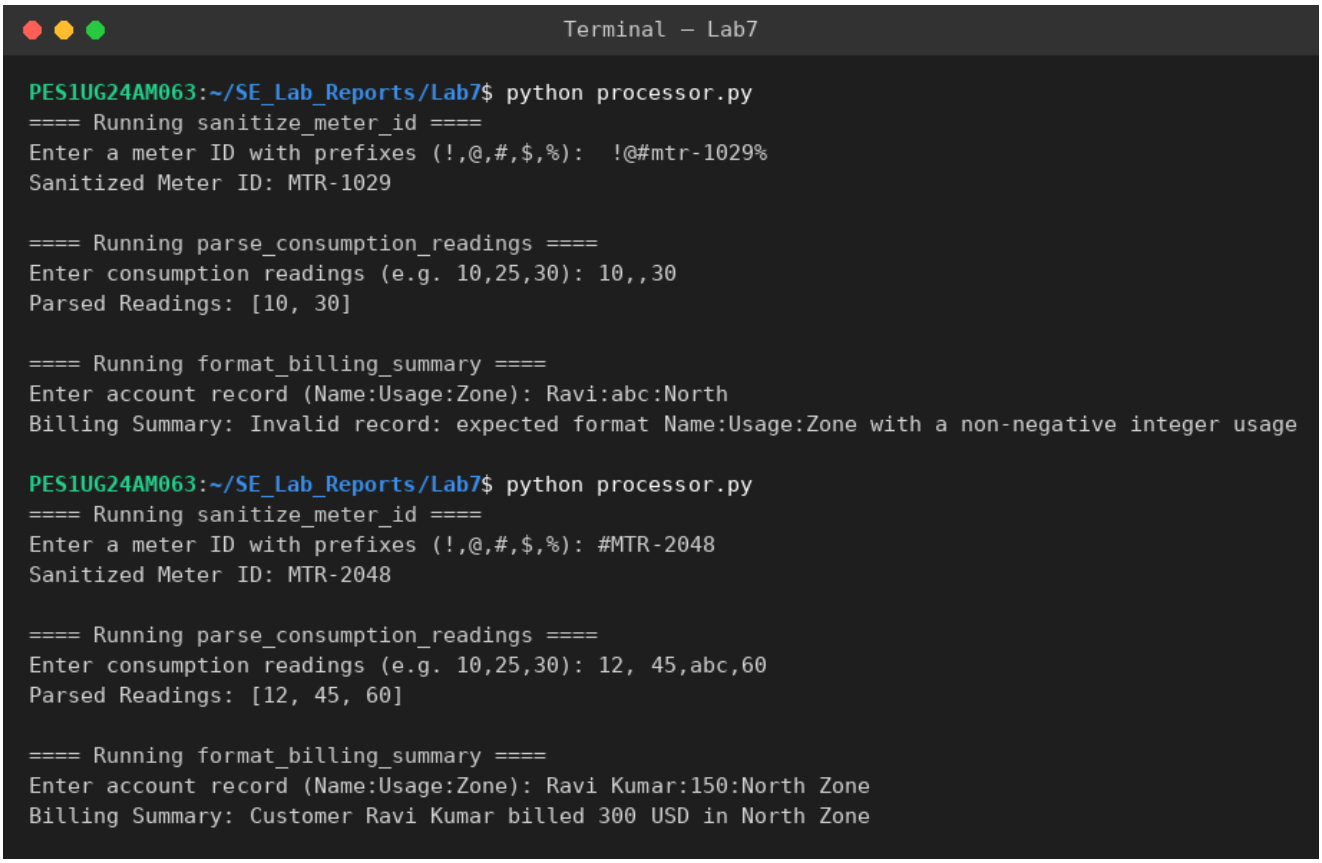
Function	Minimal failing input found	Exception
`sanitize_meter_id`	None	AttributeError (.strip on None)
`parse_consumption_readings`	'' (also '1,,3')	ValueError (int(''))
`parse_consumption_readings`	None	AttributeError (.split on None)
`format_billing_summary`	''	IndexError (parts[1] missing)
`format_billing_summary`	':' / '::'	ValueError (int(''))
`format_billing_summary`	None	AttributeError (.split on None)

Step 5: Fixing processor.py

All three functions were hardened with defensive input validation so that no input can raise an unhandled exception:

Function	Fix applied
`sanitize_meter_id`	Return '' for non-string input; strip whitespace after removing the special symbols so no leading/trailing spaces remain.
`parse_consumption_readings`	Return [] for non-string input; strip each token; skip empty and non-integer tokens instead of crashing.
`format_billing_summary`	Return an 'Invalid record' message for non-string input, wrong number of fields, or non-integer / negative usage; default empty name/zone to 'Unknown'.

Running the fixed script (`python processor.py`) with the same inputs that crashed it before, and then with valid inputs:



Step 6: Re-running the fuzz tests on the fixed code

Command: `pytest test_processor.py`. All 5 property-based tests now pass (500 generated examples each for the three main tests).

